Formation Data Engineer — RNCP 40167

Brief Apache Spark

Pipeline ETL — TradeCorp International

Docker · PySpark · Parquet · PostgreSQL · Programmation modulaire · Pytest

Informtion : commande a lancer pour acceder au notebook jupyter du conteneur docker sur le cmd :

docker logs tradecorp_training_spark 2>&1 | findstr token

puis récupérer le lien suivant :



Contexte — TradeCorp International

TradeCorp International est une entreprise de négoce fondée en 1990 qui distribue des produits alimentaires spécialisés dans 21 pays. Elle emploie 500 personnes et gère un catalogue de 77 produits répartis en 8 catégories, 29 fournisseurs et plus de 800 clients professionnels dans le monde entier.

Historiquement, TradeCorp gère ses données dans une base MySQL legacy héritée des années 2000. Chaque nuit, un script maison exporte les tables en fichiers CSV déposés sur un serveur FTP. Ces fichiers représentent l'intégralité de l'activité commerciale.

Le problème :

- Les fichiers CSV arrivent chaque nuit avec des données sales : encodages incorrects, valeurs manquantes, doublons
- L'équipe data retraite ces fichiers manuellement dans Excel chaque matin : 3 heures de travail quotidien
- Aucune automatisation, aucun historique, aucune traçabilité
- L'infrastructure est différente sur chaque poste 'ça marche chez moi mais pas chez toi'

La mission :

En tant que Data Engineer mandaté par TradeCorp, vous devez centraliser les échantillons de données dans un Datalake Azure, concevoir et implémenter un pipeline ETL automatisé qui conteneurise toute l'infrastructure avec Docker, traite les fichiers CSV avec Apache Spark, charge les données au format Parquet dans le datalake Azure.

Les données disponibles

Fichier CSV	Description	Nb lignes
customers.csv	Clients professionnels dans 21 pays	91
orders.csv	Commandes sur 3 ans (1996-1998)	830
order_details.csv	Lignes de commandes avec prix et quantités	2 155
products.csv	Catalogue produits avec stocks et prix	77
categories.csv	Catégories de produits	8
suppliers.csv	Fournisseurs dans 16 pays	29
employees.csv	Employés commerciaux	9
shippers.csv	Transporteurs	3

- Environnement : Docker + JupyterLab + PySpark
- Dataset : TradeCorp (données Northwind en CSV)
- Versioning : dépôt GitHub avec README

Structure du projet

```
tradecorp/
├── docker-compose.yml      # infrastructure
├── data/                    # CSV bruts
│   ├── customers.csv
│   ├── orders.csv
│   └── ...
├── notebooks/               # exploration Spark
│   ├── notebooks.ipynb
```

```
└── README.md
```

Notebook 1 — Exploration et prise en main de Spark

> *Créer un notebook 01_exploration.ipynb dans le dossier notebooks/. Toutes les questions de ce jalon se font dans ce notebook.*

Q1 — Créer la SparkSession

Créer une SparkSession avec le nom d'application 'TradeCorp ETL'. Afficher la version de Spark.

Q2 — Lire les 8 CSV

Lire les 8 fichiers CSV. Stocker chaque DataFrame dans une variable.

Q3 — Explorer le schema

Pour chaque DataFrame, afficher le schema des données. Identifier les types de colonnes inférés automatiquement.

Q4 — Afficher les données

Pour chaque DataFrame, afficher les 5 premières lignes. Observer la structure des données.

Q5 — Compter les lignes

Compter le nombre de lignes de chaque DataFrame avec. Créer un tableau récapitulatif.

Q6 — Statistiques descriptives

Sur df_orders et df_products essayer d'obtenir les statistiques suivantes : min, max, mean, stddev.

Q7 — Lazy evaluation

Expliquer dans une cellule Markdown ce qu'est la lazy evaluation dans Spark. Quelle est la différence entre une transformation et une action ? Donner 3 exemples de chaque.

Q8 — Spark UI

Ouvrir http://localhost:4040 dans le navigateur. Observer les jobs exécutés. Identifier ce que représentent les stages et tasks.

Q9 — Spark vs Pandas

Lire le fichier orders.csv avec Pandas et mesurer le temps. Puis avec Spark. Comparer. Expliquer quand Spark est plus pertinent que Pandas.

Q10 — Colonnes et types

Afficher la liste des colonnes de df_orders avec df.columns. Afficher les types avec df.dtypes. Identifier les colonnes qui nécessitent un cast de type.

Notebook 2 — Nettoyage et transformation des données

> *Créer un notebook 02_nettoyage.ipynb. Ce jalon couvre le nettoyage et les transformations colonne par colonne.*

Q11 — Valeurs nulles
Pour chaque DataFrame, compter le nombre de valeurs nulles par colonne. Utiliser une boucle et df.filter(col(c).isNull()).count().

Q12 — Supprimer les nulls
Dans df_orders, supprimer les lignes où shipped_date est null (commandes non livrées). Dans df_products, remplacer les valeurs nulles de unit_price par la médiane.

Q13 — Cast des types
Dans df_orders, caster order_date, required_date et shipped_date en type DateType. Dans df_order_details, caster unit_price en DoubleType et quantity en IntegerType.

Q14 — Nettoyage des chaînes
Dans df_customers, appliquer TRIM sur toutes les colonnes texte. Mettre contact_name en title case avec initcap(). Mettre country en majuscules avec upper().

Q15 — Renommer les colonnes
Dans df_order_details, renommer unit_price en prix_unitaire et quantity en quantite. Dans df_orders, renommer ship_via en shipper_id.

Q16 — Colonnes calculées
Dans df_order_details, ajouter une colonne sous_total = prix_unitaire * quantite * (1 - discount). Arrondir à 2 décimales avec round().

Q17 — Colonnes conditionnelles
Dans df_products, ajouter une colonne en_stock (True si units_in_stock > 0). Dans df_orders, ajouter une colonne is_shipped (True si shipped_date n'est pas null).

Q18 — Doublons
Vérifier s'il y a des doublons dans df_customers sur customer_id. Utiliser distinct() et count() pour comparer. Supprimer les doublons si nécessaire.

Q19 — Filtrage
Filtrer df_orders pour ne garder que les commandes de 1997. Filtrer df_products pour ne garder que les produits en stock (units_in_stock > 0) et non discontinués.

Q20 — Sélection de colonnes
Dans df_employees, sélectionner uniquement : employee_id, first_name, last_name, title, hire_date, city, country. Créer une colonne full_name = first_name + ' ' + last_name.

Écrire les DataFrames nettoyés en Parquet : PATH = "/home/jovyan/data/tmp"

Notebook 3 — Jointures et agrégations

Créer un notebook 03_transformations.ipynb. Utiliser les DataFrames nettoyés du jalon 2.

Q21 — Jointure orders + customers

Joindre df_orders et df_customers sur customer_id. Garder uniquement : order_id, company_name, country, order_date, freight.

Q22 — Jointure order_details + products

Joindre df_order_details et df_products sur product_id. Ajouter les colonnes product_name, category_id, unit_price depuis products.

Q23 — Jointure products + categories

Joindre df_products et df_categories sur category_id pour enrichir chaque produit avec category_name et description.

Q24 — DataFrame enrichi complet

A - Réaliser une première jointure complète (order_details, orders, customers, products enrichi avec categories, employees, shippers) sans renommer aucune colonne. Lister ensuite les colonnes qui apparaissent en double grâce à Counter.

B - Pour chaque colonne identifiée en Q24a, la renommer dans sa table d'origine avant de refaire la jointure, en la préfixant selon la table (customer_country, employee_country, shipper_name...). Reconstruire ensuite df_orders_enriched avec ces tables renommées, puis vérifier qu'il ne reste plus aucun doublon.

Q25 — CA par client

Calculer le chiffre d'affaires total par client (company_name) depuis df_orders_enriched. Trier par CA décroissant. Afficher le top 10.

Q26 — CA par catégorie

Calculer le CA total par catégorie de produits. Afficher le nombre de produits distincts vendus par catégorie.

Q27 — CA par mois

Calculer le CA mensuel. Utiliser date_trunc ou month() et year() pour extraire le mois et l'année.

Q28 — Performance par employé

Calculer pour chaque employé (full_name) : le nombre de commandes traitées, le CA total généré et le délai moyen de livraison en jours.

Q29 — Window functions — Rang

Classer les produits par CA généré avec dense_rank(). Utiliser une Window partitionnée par category_name.

Indice : from pyspark.sql.window import Window / from pyspark.sql.functions import dense_rank / Window.partitionBy('category_name').orderBy(desc('ca'))

Q30 — Window functions — Cumul

Calculer le CA cumulé par mois (ordre chronologique) avec sum() sur une Window orderBy date.

Q31 — Tri et limite

Afficher les 5 produits les plus vendus en quantité (toutes commandes confondues).
Afficher les 3 pays clients (customer_country) qui génèrent le plus de chiffre d'affaires.

Q32 — Écriture en Parquet

Écrire df_orders_enriched en format Parquet dans /home/jovyan/data/output/orders_enriched.parquet. Utiliser le mode overwrite.

Notebook 4 — Écriture Parquet (et chargement PostgreSQL bonus)

Q33 — Relire le Parquet

Relire le fichier Parquet et vérifier que le nombre de lignes est identique à l'original. Afficher le schema — observer que les types sont préservés.

Q34 — Comparer CSV vs Parquet

Comparer la taille du fichier CSV original vs le fichier Parquet. Observer le gain de compression. Expliquer pourquoi Parquet est plus efficace.

Q35 — Partitionnement

Écrire le DataFrame partitionné par country avec partitionBy('country'). Observer la structure des dossiers créés.

Q36 — Chargement PostgreSQL via JDBC

Ajouter PostgreSQL au docker-compose. Écrire df_orders_enriched dans PostgreSQL via JDBC dans une table orders_enriched.

Q37 — Vérifier dans pgAdmin

Ouvrir pgAdmin, vérifier que la table orders_enriched existe et contient les données. Exécuter une requête SQL de validation.

> ❗ ⚠ *Pour le JDBC vous aurez besoin du driver PostgreSQL JAR. Ajouter au docker-compose : SPARK_OPTS=--packages org.postgresql:postgresql:42.7.0*

Notebook 5 — Programmation modulaire et tests unitaires *(Jour 4 après-midi + Jour 5)*

> *On sort des notebooks. Créer des fichiers .py dans le dossier src/. Ces fichiers seront exécutés avec spark-submit depuis le conteneur Docker.*

5.1 — Les modules Python

Q38 — Créer reader.py

Créer src/reader.py avec une fonction read_csv(spark, path, filename) qui lit un CSV et retourne un DataFrame. Tester en l'important dans un notebook.

Indice : def read_csv(spark, path, filename): return spark.read.csv(f'{path}/{filename}', header=True, inferSchema=True)

Q39 — Créer transformer.py

Créer src/transformer.py avec les fonctions : clean_customers(df), clean_orders(df), clean_order_details(df), add_sous_total(df). Chaque fonction applique les transformations du jalon 2.

Q40 — Créer writer.py

Créer src/writer.py avec les fonctions : write_parquet(df, path) et write_postgres(df, table_name, jdbc_url, properties).

Q41 — Créer pipeline.py

Créer src/pipeline.py qui importe reader, transformer et writer, et orchestre tout le pipeline de bout en bout.

Indice : from reader import read_csv / from transformer import clean_orders

Q42 — Lancer avec spark-submit

Exécuter le pipeline depuis le terminal Docker : docker exec tradecorp_spark spark-submit /home/jovyan/src/pipeline.py. Vérifier que tout tourne.

Indice : docker exec -it tradecorp_spark spark-submit /home/jovyan/src/pipeline.py

5.2 — Tests unitaires avec pytest

Q43 — Installer pytest

Vérifier que pytest est disponible dans le conteneur : docker exec tradecorp_spark pytest --version. Si non, l'installer.

Indice : docker exec tradecorp_spark pip install pytest

Q44 — Créer test_transformer.py

Créer tests/test_transformer.py. Écrire un test qui vérifie que clean_orders() supprime bien les lignes où shipped_date est null.

Indice : from pyspark.sql import SparkSession / spark = SparkSession.builder.getOrCreate() / df_test = spark.createDataFrame(...)

Q45 — Test sur sous_total

Écrire un test qui vérifie que add_sous_total() calcule correctement le sous_total. Créer un petit DataFrame de test avec des valeurs connues et vérifier le résultat.

*Indice : Valeur attendue : 10.0 * 2 * (1 - 0.1) = 18.0*

Q46 — Test sur clean_customers

Écrire un test qui vérifie que clean_customers() applique bien le trim et l'initcap sur contact_name.

Q47 — Lancer pytest

Lancer tous les tests depuis le conteneur : docker exec tradecorp_spark pytest /home/jovyan/tests/ -v. Tous les tests doivent passer.

Indice : docker exec tradecorp_spark pytest /home/jovyan/tests/ -v --tb=short

> 💡 *✓ VALIDATION JALON 5 : spark-submit pipeline.py tourne sans erreur, tous les tests pytest passent. Montrer au formateur avant de continuer.*

Livrables et évaluation

Livrables attendus

- Dépôt GitHub avec le projet complet (notebooks, src/, tests/, docker-compose.yml)
- 3 notebooks fonctionnels (exploration, nettoyage, transformations)
- 4 modules Python (reader.py, transformer.py, writer.py, pipeline.py)
- Tests unitaires pytest — tous les tests passent
- Fichiers Parquet générés dans data/output/
- README.md avec instructions pour lancer le projet
- Capture d'écran de spark-submit pipeline.py qui tourne
- Capture d'écran de pytest avec tous les tests en vert
- Capture d'écran du Spark UI (localhost:4040)

Critère	Description
Exploration	8 DataFrames chargés, schemas explorés, Spark UI consulté
Nettoyage	Types corrigés, nulls gérés, colonnes calculées, doublons supprimés
Jointures & agrégations	DataFrame enrichi, agrégations, window functions
Parquet & PostgreSQL	Fichiers Parquet créés, données dans PostgreSQL
Programmation modulaire	4 modules Python fonctionnels, spark-submit tourne
Tests unitaires	Tests pytest pertinents, tous en succès
README & GitHub	Instructions claires, dépôt propre
TOTAL	

Code RNCP	Compétence	Comment couv
C25	Data Lake	Écriture Parquet partitionné
C26	Scalabilité	Spark distribué vs Pandas
C28	Pipelines de données	Pipeline ETL complet
C30	Infrastructure	Docker Compose orchestré
C34	Transformation	Nettoyage, jointures, agrégations
C38	Tests	Tests unitaires pytest sur les transformations